PROJECT DOCUMENTATION

Notivexa

AI Academic Intelligence Platform

A practical guide to running the project and understanding its frontend, backend, algorithms, technologies, and source files.

Project at a glance

Notivexa is a web application for turning a student's PDF study material into notes, summaries, flashcards, exam questions, slide content, audio lectures, and other study aids. A React frontend communicates with a Node.js/ Express backend. The backend stores account and study data in MongoDB, writes uploaded/generated files to disk, and can call hosted AI and speech services when configured.

This guide documents the project currently present in notivexa_updated. It explains the implemented code paths rather than assuming that every feature labelled "AI" uses an AI model.

Contents

- 1. Running the application
- 2. End-to-end request and data flow
- 3. Frontend structure and page responsibilities
- 4. Backend structure, API routes, and persistence
- 5. Algorithms and processing methods
- 6. Technology stack
- 7. Source-file-by-file guide
- 8. Configuration, limitations, and operational notes

1. Running the application

Run the backend and frontend in separate PowerShell terminals. Run MongoDB locally or configure a MongoDB connection URI before using database-dependent features.

Backend terminal

```
cd C:\Users\User\Desktop\Notivexa\notivexa_updated\backend
npm install
npm run dev
```

The backend package scripts provide npm run dev (Nodemon) and npm start (plain Node). The server defaults to

port 5000.

Frontend terminal

```
cd C:\Users\User\Desktop\Notivexa\notivexa_updated\frontend
npm install
npm run dev
```

Open http://localhost:3000. Vite runs the frontend on port 3000 and forwards /api and /uploads requests to http://localhost:5000. The frontend package also defines npm run build for a production bundle.

Database, demo login, and optional keys

- MongoDB defaults to mongodb://localhost:27017/notivexa. The backend reads MONGODB_URI when supplied. Without a database connection the HTTP server may still start, but accounts, documents, flashcards, chat sessions, and plans need MongoDB.
- After a successful MongoDB connection, the seed utility creates/resets student@demo.com and teacher@demo.com with password demo1234.
- Text-generation keys are optional: GEMINI_API_KEY, GROQ_API_KEY, and ANTHROPIC_API_KEY. Without them, supported generators use the local PDF/text heuristics.
- Optional speech/translation settings include GOOGLE_TTS_API_KEY, ELEVENLABS_API_KEY, and GOOGLE_TRANSLATE_KEY. Microsoft Edge TTS is also attempted without a configured API key.
- Other backend settings include PORT, NODE_ENV, JWT_SECRET, JWT_EXPIRES_IN, MAX_FILE_SIZE, and FRONTEND_URL. Store real credentials privately in backend/.env; never publish them.

2. End-to-end request and data flow

```
Browser (React pages)
  -> Axios client adds Bearer token and calls /api/...
  -> Vite development proxy forwards to Express on port 5000
  -> Express route selects middleware and controller
  -> MongoDB stores users, documents, outputs, cards, chats, and plans
  -> Local services process PDF/text, create PDFs/audio/video
  -> JSON response returns to React; /uploads serves saved files
```

Typical PDF-to-study-material flow

The following describes the primary workflow from user action through stored result.

- The user signs in. The frontend stores the returned JWT and user summary in localStorage.
- The user selects one or more PDFs. Multer checks MIME type, file count, and size, then writes the files under a user-specific backend/uploads/pdfs directory.
- The document controller calls pdf-parse via pdfEngine.js, normalizes extracted text, estimates sections and counts, and saves document metadata and full extracted text in MongoDB.
- The user selects a ready document (or section) and an output in AI Notebook. The controller verifies that the document belongs to that user and passes its text to aiService.js.
- The AI service tries configured language-model providers in order. On unavailable/failed results it uses a corresponding local PDF-engine generator where implemented.
- Some output types are rendered to PDFs through PDFKit; audio can be synthesized by a configured speech provider; video uses FFmpeg.
- The backend returns content and download URLs. React renders Markdown/Math or media and the Express static /uploads route serves the files.

3. Frontend structure and page responsibilities

The frontend is a React 18 single-page application using React Router. Shared authentication and theme state are React contexts. The common Shell wraps authenticated pages with the sidebar and header. Axios is the shared HTTP client.

frontend/src/main.jsx - React entry point; installs BrowserRouter, ThemeProvider, App, and toast notifications.

frontend/src/App.jsx - Routes; protects app pages and redirects unauthenticated users to login.

frontend/src/context/AuthContext.jsx - Login/register/logout state; persists token and user in localStorage.

frontend/src/context/ThemeContext.jsx - Default/light/dark theme state and document data-theme persistence.

frontend/src/components/Layout/Shell.jsx - Authenticated sidebar, route navigation, header, user details, and theme cycle.

frontend/src/utils/api.js - Axios base URL, JWT Authorization header, and 401/server-error handling.

frontend/src/index.css - Theme variables and reusable app layout, cards, inputs, buttons, tables, and responsive styles.

frontend/index.html - HTML host page and web-font loading (Inter, Sora, Caveat).

Page behavior

pages/Login.jsx - Email/password login, demo buttons, success/error messages.

pages/Register.jsx - Account registration form and role/institution inputs.

pages/Dashboard.jsx - Student stats, feature shortcuts, and API setup status/help.

pages/Upload.jsx - PDF drag/drop and file picker, upload progress, extraction status, and navigation to tools.

pages/Notebook.jsx - Document/section/output/language selection, generation request, Markdown/KaTeX rendering, and downloadable results.

pages/Flashcards.jsx - Flashcard set selection, flip cards, study flow, difficulty and mastered indicators; sample cards appear if none are saved.

pages/Podcast.jsx - Document and voice/language selection, podcast generation, script display, and audio player.

pages/Chatbot.jsx - Document selection, suggested questions, message submission, and chat display.

pages/Library.jsx - Document and generated-output lists, search, download links, and document deletion.

pages/Ebooks.jsx - Curated books, external learning resources, and MCA/video resource lists.

pages/Planner.jsx - Exam date, subject/hour selection, generated tasks, progress, completion, and static subject question suggestions.

pages/Settings.jsx - Profile editing, password change, account stats, and theme selection.

frontend/vite.config.js - Vite React plugin, dev port 3000, and backend proxy mappings.

frontend/package.json - Vite scripts and React/frontend dependency declarations.

Frontend route map

```
/login       -> Login
/register    -> Register
/dashboard   -> Dashboard
/upload      -> Upload
/notebook    -> AI Notebook
/flashcards  -> Flashcards
/podcast     -> Audio Lecture
/chatbot     -> Ask Your PDF
/library     -> My Library
/ebooks      -> E-Books and resources
/planner     -> Study Planner
/settings    -> Settings
```

The route definitions are in frontend/src/App.jsx. A user-facing page often calls api.get/api.post and then updates React state with the returned data. The shared client sets a long request timeout (180 seconds), appropriate for some generation requests.

4. Backend structure, API routes, and persistence

The backend is a CommonJS Express application. server.js loads environment configuration, installs security/logging/body parsing middleware, mounts route modules, serves uploaded files, connects to MongoDB, and starts the HTTP listener.

API route groups

/api/auth - Registration, login, profile read/update, and password change.

/api/documents - Authenticated PDF upload, document list/read/delete, and stats.

/api/generate - Notes, summary, flashcards, questions, question papers, handwritten notes, podcast, slides, video, mind map, and vocabulary.

/api/flashcards - List/read card sets, update card status, and delete a set.

/api/chatbot - Post tutor messages and list/read/delete chat sessions.

/api/library - List user documents and generated output records.

/api/ebooks - Filter a static catalogue of book links by search and subject.

/api/planner - List, generate, update task status, and deactivate study plans.

/api/setup and /api/health - Report configured provider availability and basic server health.

Backend source modules

controllers/authController.js - Validates basic account input, hashes via the model hook, issues JWTs, and handles account changes.

controllers/documentController.js - Creates document records, extracts PDFs, saves status/text/counts, and serves user-scoped queries.

controllers/generateController.js - Loads the user-owned document or selected section; orchestrates generators and output persistence.

controllers/chatController.js - Retrieves the user-owned PDF, calls tutor response generation, and persists bounded conversation history.

middleware/auth.js - Reads and verifies Bearer JWTs and loads the active user.

middleware/upload.js - Configures Multer disk storage, PDF MIME filtering, UUID filenames, file-count and file-size limits.

models/index.js - Mongoose schemas and models for User, Document, Flashcard, Chat, and Plan.

services/aiService.js - Hosted LLM selection, prompts, translation, generation functions, and TTS fallback chain.

services/pdfEngine.js - PDF text extraction, section/fact/definition heuristics, local study-material fallbacks.

services/pdfGenerator.js - PDFKit rendering for standard notes, notebook-style notes, question papers, and slides.

services/videoEngine.js - FFmpeg rendering, aspect ratio conversion, motion effect, audio muxing, and subtitle file creation.

utils/seed.js - Demo account seeding/reset after successful MongoDB connection.

backend/package.json - Express/server scripts and backend dependency declarations.

backend/.env - Runtime settings and optional external-service keys; do not share its values.

Data models

- User: name, email, hashed password, role, institution, language preference, activation state, and study statistics.
- Document: user owner, title/original filename, disk path/type/size, processing status, full extracted text, sections, page/word counts, subject, and generated-output metadata.
- Flashcard: user/document references, title/subject/language, cards with Q&A/difficulty/isKnown, and totals.
- Chat: user/document context and timestamped user/assistant messages.
- Plan: user, exam date, subjects, daily hours, active state, and completion/priority-tagged tasks.

Persistence and ownership

Most API routes require authentication. Document and chat controllers scope queries to the authenticated user. Uploaded PDFs and generated assets are files on disk; references, extracted text, and output metadata are

stored in MongoDB. The server mounts the uploads directory as static content.

5. Algorithms and processing methods

Hosted AI generation is prompt-based, not a local trained model

aiService.js sends a system prompt plus the selected PDF text and task-specific instructions to external language-model APIs. Provider order is Gemini, Groq, then Anthropic Claude. Successful non-empty output is returned; provider errors are logged and the next configured provider is tried. The model selection is not a bespoke algorithm trained in this repository.

PDF extraction and heuristic analysis

- PDF text is read by pdf-parse. The local cleaning routine normalizes line endings/form feeds, reduces repeated whitespace, and limits retained character ranges for selected scripts.
- Section detection uses regular expressions for Chapter/Unit/Section/Module/Topic/Part headings, numbered headings, uppercase headings, and Roman numerals. A heading only begins a new section after a minimum amount of prior text; at most 50 sections are returned.
- Sentence extraction uses punctuation boundaries and a sentence-length range. Key-fact detection retains sentences containing numbers or clue words/symbols for formulas, laws, definitions, results, and conclusions.
- Definition detection uses regex patterns such as “X is defined as”, “X refers to”, and “X means”. Topic detection checks uppercase lines and recognized heading forms.
- Paragraphs are grouped into a requested number of text chunks. The local note/summary/question/podcast/PPT/mind-map/vocabulary methods use these extracted facts, definitions, topic lines, and chunks.
- Local flashcards are assembled from extracted definition/fact patterns, then padded from suitable sentences. These deterministic rules are simple fallbacks, not semantic model output.

Language translation

For selected languages, the AI service can try Google Translate when configured and otherwise calls MyMemory in short chunks with a delay. Some generator functions only invoke translation when the generated response does not appear to contain Devanagari text. This is a lightweight script check, not robust language identification.

Study planner: deterministic schedule

The planner’s “generate” route does not call the AI service or calculate an optimized schedule. It iterates over two weeks of Monday through Saturday, cycles through the submitted subjects, assigns Saturday revision tasks, derives durations from dailyHours, and sets a simple repeating priority pattern.

Audio/video processing

- Audio text is split around sentence boundaries to satisfy provider input-size constraints. TTS fallback order is Google TTS, ElevenLabs, Microsoft Edge TTS, then Google Translate’s speech endpoint.
- Video narration is split into groups of four words for subtitle cues. Cue durations are proportional to phrase character counts across the requested duration.
- FFmpeg renders a 720x1280 vertical MP4, optionally loops a background image and applies a gentle zoom, burns ASS subtitles when available, encodes H.264/AAC, and ends when audio ends.

PDF rendering

PDFKit creates A4 documents. Handwritten-style PDFs use bundled handwriting fonts where present, draw notebook rules/margins, and format recognized headings, bullets, diagrams, and tables. The output is stylized typesetting, not a scan or actual handwriting.

6. Technology stack

Frontend

- React 18 and React DOM: component UI and state.
- React Router DOM 6: client-side routes and protected-page navigation.
- Vite 5 and the Vite React plugin: development server and build.
- Axios: frontend-to-backend HTTP calls; React Hot Toast: notifications.
- React Markdown, remark-gfm, remark-math, rehype-katex, KaTeX: formatted output, tables, and mathematical notation.
- CSS with shared custom properties for default/light/dark themes; Google Fonts references for Inter, Sora, and Caveat.

Backend

- Node.js and Express 4: HTTP API and middleware.
- MongoDB and Mongoose 8: persistent document models and queries.
- jsonwebtoken and bcryptjs: JWT authentication and password hashing.
- Multer and UUID: disk upload handling and unique file names.
- pdf-parse: extracting text from PDF files; PDFKit: generating PDFs.
- fluent-ffmpeg and @ffmpeg-installer/ffmpeg: MP4 generation and encoding.
- node-fetch, dotenv, cors, helmet, morgan, express-rate-limit: outbound HTTP, configuration, cross-origin policy, security headers, request logging, and throttling.

Optional external services

Google Gemini, Groq, Anthropic Claude, Google translation/TTS, MyMemory translation, ElevenLabs, Microsoft Edge TTS, and Google Translate TTS. Availability, quota, model support, network access, and API keys affect which paths succeed.

7. Source-file-by-file guide

The list below covers authored application source and key configuration files under notivexa_updated. It excludes node_modules, generated dist bundles, lockfiles, and binary test/runtime assets.

Backend files

backend/server.js - Application startup, middleware, route mounts, health/setup routes, database connection, listener, and error responses.

backend/controllers/authController.js - Register/login/profile/password request logic.

backend/controllers/chatController.js - Tutor question handling and chat-session storage.

backend/controllers/documentController.js - Upload/extract/list/read/delete documents and calculate stats.

backend/controllers/generateController.js - Document-based content generation, PDF/audio/video orchestration, and output metadata.

backend/middleware/auth.js - JWT Bearer verification and active-user lookup.

backend/middleware/upload.js - PDF-only Multer disk storage and upload limits.

backend/models/index.js - All Mongoose schemas and model exports.

backend/routes/auth.js - Auth endpoint declarations.

backend/routes/chatbot.js - Chat message and session endpoint declarations.

backend/routes/documents.js - Upload and document CRUD/stat endpoint declarations.

backend/routes/ebooks.js - Static catalogue filtering endpoint.

backend/routes/flashcards.js - Flashcard read/update/delete endpoint declarations.
backend/routes/generate.js - Study-content generation endpoint declarations.
backend/routes/library.js - Document/output listing endpoint declarations.
backend/routes/planner.js - Study plan creation, listing, completion update, and deactivation.
backend/services/aiService.js - LLM provider calls, prompts, translation, fallbacks, audio, and content generation.
backend/services/pdfEngine.js - PDF extraction, text heuristics, and offline content generators.
backend/services/pdfGenerator.js - PDF rendering functions.
backend/services/videoEngine.js - Subtitle file generation and FFmpeg MP4 pipeline.
backend/utils/seed.js - Demo student and teacher account seed logic.
backend/package.json - Backend scripts and package dependency declarations.
backend/.env - Local runtime configuration; do not expose credential values.

Frontend files

frontend/index.html - Browser HTML shell and font preconnect/imports.
frontend/vite.config.js - Port and development proxy configuration.
frontend/package.json - Frontend scripts and package dependency declarations.
frontend/src/main.jsx - React DOM entry point and global providers.
frontend/src/App.jsx - Route table and authentication route guards.
frontend/src/index.css - Theme tokens and global component/layout styling.
frontend/src/context/AuthContext.jsx - Authentication state and persistence.
frontend/src/context/ThemeContext.jsx - Theme selection and persistence.
frontend/src/components/Layout/Shell.jsx - Navigation shell and shared header/sidebar.
frontend/src/utils/api.js - Axios configuration, JWT injection, and response handling.
frontend/src/pages/Login.jsx - Login interface.
frontend/src/pages/Register.jsx - Registration interface.
frontend/src/pages/Dashboard.jsx - Dashboard, API setup prompt, and tool navigation.
frontend/src/pages/Upload.jsx - PDF upload interface and extraction results.
frontend/src/pages/Notebook.jsx - Generation controls and output rendering.
frontend/src/pages/Flashcards.jsx - Card sets, flip-card interaction, and study mode.
frontend/src/pages/Podcast.jsx - Audio generation configuration, player, and script view.
frontend/src/pages/Chatbot.jsx - Document-aware tutor chat UI.
frontend/src/pages/Library.jsx - Document/output inventory and downloads.
frontend/src/pages/Ebooks.jsx - Curated external learning-resource browser.
frontend/src/pages/Planner.jsx - Plan generation controls, task progress, and questions.
frontend/src/pages/Settings.jsx - Profile/password/theme settings.

Other directories and assets

• backend/uploads/: user PDFs and generated PDF, audio, and video files at runtime. These are data/output files, not source code.
• backend/*.ttf and frontend font imports: handwriting/visual font assets used by generated documents or UI.
• backend/bg.jpg and other bundled media/test PDFs/audio/MP4s: assets or local test artifacts; they are not part of the main application logic.
• frontend/dist/: generated production build output; rebuild it from frontend source with npm run build.
• backend/node_modules/ and frontend/node_modules/: installed third-party libraries; do not treat their files as project-authored source.
• package-lock.json files: locked dependency trees for repeatable npm installation.

8. Configuration, limitations, and operational

notes

- Scanned/image-only PDF support: the current upload path filters for PDFs and the extractor expects embedded readable text. There is no OCR pipeline in the reviewed path.
- General chatbot option: the interface offers a no-document/general tutor selection, but answerChat currently requires document text and asks the user to choose a PDF when it is absent.
- Fallback quality: local generators use pattern matching and snippets, so they cannot guarantee complete understanding, factual correctness, balanced questions, or true exam-quality outputs.
- Planner label: the generated schedule is deterministic and template-based, not model-generated or an optimized timetable.
- Video generation depends on a usable audio file and FFmpeg. The generator's final error/fallback handling should be considered when diagnosing a missing video.
- Security configuration: use a strong private JWT_SECRET outside development. The code has a development fallback secret, so production deployment should explicitly set a secret.
- MongoDB status: a listening server is not proof that MongoDB-backed functionality is ready. Confirm the database connection log and /api/health response.
- Generated files are saved on the server filesystem; back up both MongoDB data and the relevant uploads directory if preserving user data is required.

Key files to start reading

```
frontend/src/App.jsx
frontend/src/utils/api.js
backend/server.js
backend/routes/documents.js
backend/controllers/documentController.js
backend/services/pdfEngine.js
backend/controllers/generateController.js
backend/services/aiService.js
backend/models/index.js
```

This PDF is an implementation walkthrough based on the source structure. It does not include environment-variable values or claim that the application has been started or tested in this session.